

AgentCore Memory Architecture Guide

Strands Framework · EU Banking · Session Resume Patterns

Updated with latest releases (GA, stateful runtime Feb 2026, Lambda MicroVMs Jun 2026), conversation sidebar architecture, microVM lifecycle, and the definitive guide to how agents resume when short-term memory has expired.

Short-Term Memory

Long-Term Memory

Session Resume

MicroVM Lifecycle

GDPR / DORA

Token Optimisation

Sidebar Architecture

Terraform IaC

VERSION	DATE	STATUS	CLASSIFICATION
3.0	June 2026	PRODUCTION	CONFIDENTIAL

Table of Contents

1.	What's New — Release Timeline (2025–2026)
2.	Executive Summary
3.	AgentCore Architecture — Core Concepts
3.1	Memory Resource, Events & Namespaces
3.2	Five Design Principles
3.3	MicroVM Session Lifecycle
3.4	Memory Lifecycle Flow
4.	Memory Types — Complete Taxonomy
4.1	Short-Term Memory
4.2	Long-Term Memory Strategies
4.3	Episodic Memory
4.4	Persistent Session Storage (/mnt/workspace)
4.5	Retention Period Decision Matrix
5.	Conversation History Sidebar — Architecture
5.1	How Claude / ChatGPT Build Their Sidebars
5.2	The Three Layers of Session State
5.3	Reimagining the Sidebar on AWS AgentCore
5.4	Session Browser — Complete AWS Reference Architecture
6.	Session Resume — The Full Lifecycle
6.1	Scenario A: Resume Within Idle Timeout (Warm Start)
6.2	Scenario B: Resume After Idle Timeout, STM Still Alive
6.3	Scenario C: Resume After STM Expiry — Cold Reconstruction
6.4	Scenario D: Resume After Both STM and LTM Are Unavailable
6.5	Decision Tree — Which Resume Path to Use
7.	Multi-Agent Memory Patterns — Pros & Cons
8.	Memory Processors & Extractors
9.	Framework Comparison — Pros & Cons
10.	Memory & Token Optimisation Strategies
11.	Strands Framework Best Practices
12.	EU Banking, GDPR & Regulatory Compliance
13.	Security, Policy & Threat Model
14.	Cost Analysis & Optimisation
15.	Project Journey — PoC to Production
16.	Evaluation Framework
17.	Terraform IaC Reference
18.	Risks, Recommendations & Decision Guide

1. What's New — Release Timeline (2025–2026)

Validated against AWS docs, re:Post articles, and official release announcements

AgentCore has moved extremely fast since its announcement. This section captures every significant release validated against official AWS documentation and trusted community sources as of June 2026. All subsequent chapters reflect these updates.

Jul 2025	AWS Summit NY Announcement	AgentCore Memory announced. Short-term + Long-term memory, Strands SDK integration revealed.
Aug 2025	AWS Blog Published	Official deep-dive blog: memory resource, events, strategies, retrieval API documented publicly.
Oct 2025	General Availability (GA)	All AgentCore services GA. VPC, PrivateLink, CloudFormation, resource tagging added. A2A protocol support in Runtime. Self-managed memory strategy added. 9 AWS regions including eu-central-1 and eu-west-1.
Dec 2025	AgentCore Policy Preview	Natural language → Cedar policy enforcement. AgentCore Evaluations (13 evaluators + custom). Episodic memory strategy GA.
Feb 2026	Stateful Runtime Environment	Firecracker microVM per session. Persistent session storage (/mnt/workspace, S3-backed). 15-min idle timeout (configurable to 28800s). 8-hour max microVM lifetime. Warm-pool pre-warming capability.
Mar 2026	Performance Benchmarks	Pre-warmed sessions 90% faster: latency from ~2.9s to ~250ms average. AgentCore CLI ships (CDK-based IaC, Terraform coming).
Apr 2026	AgentCore CLI GA	CLI deploys agents as IaC with audit history. Harness: define model + tools + system prompt — no orchestration code. Model-agnostic: switch models mid-session. AI coding assistant skills (Claude Code, Codex, Kiro) included.
May 2026	Strictly Consistent Metadata	New: STRICTLY_CONSISTENT extraction type on LTM metadata. Up to 3 keys per strategy. Ensures metadata passes through LLM extraction unchanged. Supports semantic, preference, and episodic strategies.
Jun 2026	AWS Lambda MicroVMs	New compute primitive: dedicated Firecracker VMs per session, snapshot-based startup, suspend/resume state, up to 16 vCPU / 32 GB RAM / 32 GB disk. Complements AgentCore Runtime. Available us-east-1, eu-west-1, ap-northeast-1.

★ **Strictly Consistent Metadata (May 2026) — Key Impact:** Previously, all metadata on long-term memory records was inferred by the LLM during extraction, meaning values could drift or be misclassified. Now, setting `extraction_type=STRICTLY_CONSISTENT` guarantees the exact value you supply arrives on the record. This is critical for tenant isolation, session scoping, and multi-agent routing in banking contexts.

2. Executive Summary

Amazon Bedrock AgentCore Memory (GA October 2025, latest update June 2026) is the fully managed memory layer for AWS AI agents. It eliminates the need to manually orchestrate DynamoDB, OpenSearch, Redis, and custom retrieval logic. Built on **Firecracker microVMs** for session isolation and a purpose-built **two-tier memory architecture** (short-term events + long-term vector store), it enables agents to maintain context within a session, resume across sessions, and recover gracefully when memory has expired. This guide has been updated to cover the complete session resume lifecycle — the most commonly misunderstood aspect of production memory architectures.

Dimension	Current State (June 2026)
Memory Service GA	October 2025 — 9 AWS regions (eu-central-1, eu-west-1 for EU banking)
Runtime Architecture	Firecracker microVM per session (Feb 2026). 15-min idle timeout, 8-hour max lifetime.
Memory Tiers	Short-term: raw events, 7-365d retention. Long-term: extracted vector records, indefinite.
Session Storage	Persistent /mnt/workspace S3-backed filesystem. Survives microVM termination. 14-day idle retention.
New in 2026	Stateful runtime, strictly consistent metadata, Lambda MicroVMs, AgentCore CLI, A2A protocol
Strands SDK	AgentCoreMemorySessionManager: STM + LTM with hooks. Prompt caching CacheConfig(strategy='auto').
Resume When STM Expires	Long-term memory retrieval + session metadata (DynamoDB) reconstructs conversation context
EU Banking	CMK mandatory, VPC/PrivateLink, eu-central-1 primary, GDPR Art. 17 erasure workflow required
Token Optimisation	65% enterprise AI failures from context drift (not exhaustion). 70% compaction threshold.

3. AgentCore Architecture — Core Concepts

3.1 Memory Resource, Events & Namespaces

A **Memory Resource** is the top-level logical container. It holds raw **Events** (short-term) and extracted **Memory Records** (long-term). Organisation is via a hierarchical namespace: `/strategy/{strategyId}/actor/{actorId}/session/{sessionId}/`. IAM conditions on namespace paths enforce multi-tenant isolation. The trailing slash is mandatory to prevent prefix collisions in multi-tenant applications.

Component	Type	Stored As	Retention	Retrieval API
Memory Resource	Container	Logical (no storage cost)	Persistent	N/A — defines policies
Events	Short-term	Append-only event log	7–365 days (per config)	ListEvents, GetEvent
Memory Records	Long-term	Vector-indexed records	Indefinite until delete	RetrieveMemoryRecords
Session Namespace	Scope	IAM condition key	Persistent	namespacePath filter
Metadata (LTM)	Annotation	String key-value pairs	Same as record	Metadata filter on retrieval

3.2 Five Design Principles

Abstracted Storage	No DynamoDB, OpenSearch, Redis to manage. Single API surface. Vector store managed internally.
Security by Default	Encrypted at rest and in transit. AWS-managed or CMK. Per-session microVM isolation — complete memory sanitisation on termination.
Continuity	Events stored chronologically. Session branching for parallel tasks. Filesystem persists across microVM restarts.
Hierarchical Namespaces	actor_id / namespace / memory_id hierarchy. IAM condition keys (bedrock-agentcore:namespace, bedrock-agentcore:namespacePath) for RBAC.
Scalable Retrieval	Internally managed vector embeddings. Semantic similarity search. Strictly consistent metadata (May 2026) for deterministic filtering.

3.3 MicroVM Session Lifecycle (February 2026)

Each runtimeSessionId is bound to a dedicated Firecracker microVM. The microVM transitions through three states. Understanding this lifecycle is essential for designing the session resume architecture in section 6.

ACTIVE	microVM is processing a sync request or executing background tasks (agent reports HealthyBusy in /ping). All in-memory state and filesystem available. Model invocations billed at active rate.
IDLE	No requests being processed. microVM still provisioned — billed at idle (memory-only) rate. Responds instantly to next invocation (warm start, ~200ms p50 vs ~2.9s cold). Survives up to idleRuntimeSessionTimeout (default 900s / 15 min).
TERMINATED	microVM destroyed and memory sanitised after: (a) idle timeout expires, (b) 8-hour max lifetime reached, (c) explicit StopRuntimeSession call, or (d) health check failure. Next invocation triggers cold start (~2.9s). Session storage (/mnt/workspace) survives microVM termination for 14 days.

Lifecycle Parameter	Default	Min	Max	Notes
idleRuntimeSessionTimeout	900s (15m)	1s	28800s	Time idle before microVM terminates. Must be <= maxLifetime.
maxLifetime	28800s (8h)	1s	28800s	Hard ceiling on microVM lifetime. New session required after this.
Session storage idle	14 days	N/A	N/A	Filesystem (/mnt/workspace) retained 14 days after last activity.
LTM extraction latency	Async	N/A	N/A	Gap between writing events and LTM records being searchable.

■ **Pre-warming:** Benchmark (April 2026) shows pre-warmed sessions serve requests 90% faster than cold starts (2.9s → 250ms avg). Implement a heartbeat loop that pings idle sessions with HealthyBusy status every 10 minutes to keep them alive past the idle timeout threshold.

3.4 Memory Lifecycle Flow

1	User message arrives PIIRedactionHook fires first — redacts before any storage. ConsentCheckHook validates GDPR basis.
2	Write short-term event put_events call buffers message to AgentCore Memory event log (batch_size controls flush frequency).
3	Async extraction fires After session ends (EventBridge trigger), LLM-based extraction pipeline reads events and produces structured long-term memory records.
4	Consolidation stores LTM Extracted records stored in vector index under /strategy/{id}/actor/{actorId}/namespace. Strictly consistent metadata preserved exactly.
5	Next session opens MemoryRetrievalHook fires: semantic search over LTM records, injects top-K into system prompt before model call.
6	STM replay (if alive) If session's short-term events still within retention window, ListEvents API replays raw conversation for full context reconstruction.

4. Memory Types — Complete Taxonomy

4.1 Short-Term Memory (Working Memory)

Short-term memory stores raw conversation events as an ordered, append-only log. It is the foundation of all other memory types. The key insight: **short-term memory is not just for in-session context — it is the input data source for the LTM extraction pipeline**. Without events, no long-term memories can be extracted.

Property	Value	Notes
Retention	7 to 365 days (per Memory Resource)	Set at resource creation — cannot change without recreation
Scope	Single or cross-session	Cross-session: same actor_id links sessions across time
batch_size	1 to N messages	Use 10+ — reduces API calls 90%. Flush on session close.
Retrieval	ListEvents, GetEvent	Returns raw events in chronological order. Used for STM replay.
Branching	Supported	Parallel tasks within same actor context
Encryption	AWS-managed default or CMK	CMK mandatory for EU banking / regulated workloads
Session isolation	Complete per session_id	Enforced at service level — cannot be overridden
Expiry on LTM	Async extraction runs post-session	LTM records available after consolidation delay (not instant)

4.2 Long-Term Memory Strategies

Strategy	What It Extracts	Best For	Strictly Consistent Metadata?	GDPR Basis
SUMMARIZATION	Condensed session highlights	Advisory sessions, troubleshooting	Supported (3 keys max)	Legitimate interest
USER_PREFERENCE	Likes, dislikes, behaviour patterns	Personalisation, wealth banking	Supported	Consent required
SEMANTIC	Domain facts, entities, relationships	KYC, product knowledge, org data	Supported	Legitimate interest
EPISODIC	Episodes with context and outcomes	Pattern learning, adaptation	Supported	Legit interest + audit
SELF-MANAGED	Anything via custom Lambda pipeline	Full control, domain-specific	N/A — operator controls fully	Operator owns fully

★ **Strictly Consistent Metadata (NEW May 2026):** Set `extraction_type='STRICTLY_CONSISTENT'` on a metadata key to guarantee it passes through LLM extraction unchanged. Critical for: `tenant_id` isolation (never let LLM infer it), `session_id` scoping, `conversation_thread` linking, and compliance-grade record tagging. Up to 3 keys per strategy.

4.3 Episodic Memory

A reflection agent analyses structured episodes to extract reusable patterns. When similar tasks arise, the main agent retrieves learnings. AWS demonstrated at `re:Invent`: agent learned from solo trip booking, then automatically adjusted for a family trip three months later. Requires quarterly EBA fairness audit for EU banking.

■ **EU Banking Warning:** Episodic memory influencing credit or fraud must provide human review pathway (GDPR Art. 22). Reflection reasoning must be logged (MiFID II). EBA ML Guidelines §5.1 requires quarterly demographic fairness audit.

4.4 Persistent Session Storage (/mnt/workspace) — NEW Feb 2026

A durable, S3-backed POSIX filesystem mounted at **/mnt/workspace** inside the microVM. Unlike short-term memory (conversation events), session storage holds files, code, build artifacts, and working state. It survives microVM termination and is re-mounted when a new microVM picks up the same runtimeSessionId. Retained for 14 days of inactivity.

Capability	Short-Term Memory	Session Storage (/mnt/workspace)
What it stores	Conversation events (text messages)	Files, code, models, database files, build artifacts
Survives microVM stop?	YES (in AgentCore Memory service)	YES (S3-backed — persists for 14 days after last activity)
Survives session end?	YES (until retention TTL expires)	YES (until 14-day idle expiry or explicit cleanup)
Retrieval method	ListEvents API — chronological	POSIX filesystem (ls, cat, git, npm — standard tools)
Indexing	None — raw event log	None — file system traversal
Primary use	LTM extraction input + STM replay	Coding agents, data pipelines, long-horizon work sessions
Best practice	batch_size=10, flush on close	Write to /mnt/workspace; session auto-syncs to S3

4.5 Retention Period Decision Matrix

Use Case	Memory Type	Retention	GDPR Lawful Basis	Regulatory Driver
Customer support chat	Short-term	7 to 30 days	Legitimate interest	CX best practice
Loan application	STM + Checkpoint	90 days	Contract performance	Dispute resolution
Wealth management prefs	LTM (Preference)	2 years	Consent	MiFID II suitability
Fraud investigation	LTM (Semantic)	7 years	Legal obligation	5AMLD / Art. 6(c)
Trading preferences	LTM (Preference)	1 year	Consent + Legal	MiFID II Art. 25
KYC entities	LTM (Semantic)	5 years	Legal obligation	4AMLD / 5AMLD
Episodic patterns	Episodic	1yr + review	Legitimate interest	EBA ML Guidelines
Session workspace files	Session Storage	14 days idle	Contract performance	Functional continuity

5. Conversation History Sidebar — Architecture

How Claude and ChatGPT build their sidebars, and how to re-imagine this on AWS AgentCore

5.1 How Claude / ChatGPT Build Their Sidebars

The conversation history sidebar is one of the most used features in consumer AI products. Understanding how it is built reveals the three fundamental data requirements every production agent application must satisfy — and maps directly to what AgentCore provides.

Component	What It Stores	How It's Used	Claude Implementation
Conversation Index	Session ID, title, timestamp, last message preview	Browse sidebar, search past conversations	JSONL transcript index per project directory
Conversation Transcript	Full message history (user + assistant turns)	Resume: full context re-injected into model window	*.jsonl per session_id in ~/.claude/projects/
Session Metadata	Branch/fork graph, labels, project, model used	Resume from fork, filter by project, show metadata	sessions-index.json
Cross-Session Memory	User preferences extracted from past chats	Injects personalisation into new conversations via RAG	settings.json + memory feature (RAG over history)
Search Index	Embeddings or keyword index over past messages	Find specific past conversation by content	RAG tool call visible in conversation

Key architectural insight from Claude Code's implementation: **session state is written to disk on every event, not on exit** — so a crash loses nothing. Sessions are stored as JSONL files (one event per line) with unique session IDs. The sidebar is built entirely from file reads; no running agent is required. This is the pattern to replicate on AWS.

★ **Claude's memory feature (2025):** Chat search uses RAG over conversation history — past conversations are indexed, and the agent performs a tool call to retrieve relevant context from prior sessions. Memory is updated within 24 hours of conversation changes. Users can view, edit, import, and export memories from Settings > Capabilities.

5.2 The Three Layers of Session State

Any production conversation sidebar requires three distinct layers. Conflating them leads to either data loss (too little persistence) or cost and compliance problems (too much).

Layer	Purpose	Lifespan	Storage Mechanism	AgentCore Equivalent
1. Session Catalog	List conversations, titles, timestamps, metadata for sidebar display	Permanent (user-controlled)	Low-latency key-value store per user	DynamoDB: conversations table (actor_id, session_id, title, last_updated, metadata)
2. Conversation Transcript	Full message history for exact resumption of context	Session TTL (days to years)	Append-only durable log	AgentCore STM: events API (ListEvents, GetEvent). Expires per event_expiry_duration config.
3. Extracted Knowledge	User preferences, facts, summaries — portable across sessions	Indefinite until deleted	Vector-indexed semantic store	AgentCore LTM: memory records (RetrieveMemoryRecords). Survives STM expiry permanently.

5.3 Reimagining the Sidebar on AWS AgentCore

Below is the complete AWS architecture for a Claude-style conversation history sidebar, using AgentCore Memory as the backbone. Every layer maps to a specific AWS service with clear responsibilities and data contracts.

Frontend Sidebar (React / React Native)	Displays conversation list from DynamoDB. Renders title, timestamp, last-message preview, labels. Triggers resume on click. Allows rename, fork, delete, search. Connects via AppSync or REST API Gateway. <i>Implementation: AppSync / API Gateway → Lambda → DynamoDB (session catalog)</i>
Session Catalog (DynamoDB)	Stores: actor_id (PK), session_id (SK), title, created_at, last_updated_at, model_id, preview_text (first 120 chars of last message), labels[], is_pinned, parent_session_id (for forks). GSI on actor_id + last_updated_at for sorted sidebar rendering. <i>Implementation: DynamoDB: Global Secondary Index on actor_id + last_updated_at</i>
Transcript Store (AgentCore STM)	Stores every event (user + assistant + tool calls) via put_events. Retrieved via ListEvents for full context reconstruction on resume. Expires per event_expiry_duration (30–365 days). The event log IS the canonical conversation record while it lives. <i>Implementation: AgentCore Memory: events API. batch_size=10. runtimeSessionId maps to DynamoDB session_id.</i>
Knowledge Store (AgentCore LTM)	Extracted preferences, facts, summaries — survives STM expiry. Retrieved semantically via RetrieveMemoryRecords. Injected as RAG context at session start. Used for reconstruction when STM has expired. STRICTLY_CONSISTENT metadata tags actor_id and session_id onto records. <i>Implementation: AgentCore Memory: LTM strategies. Extraction fires via EventBridge post-session trigger.</i>
Session Search (Kendra / OpenSearch)	Optional: semantic search over conversation content. Indexes transcript text when STM events are written. Enables 'find my conversation about X' queries in sidebar search bar. For EU banking: encrypt and keep in eu-central-1. <i>Implementation: Amazon Kendra or OpenSearch Serverless (same region as AgentCore Memory)</i>
Cross-Session Summary (S3 + Lambda)	After session close, Lambda exports compressed conversation summary to S3 (JSON). This is the permanent audit archive — survives after both STM and LTM TTLs expire. Used for compliance, SAR (GDPR Art. 15), and ultra-long-horizon reconstruction. <i>Implementation: S3 with server-side KMS encryption. Object tags: actor_id, session_id, date.</i>

5.4 Session Browser — Complete AWS Reference Architecture

```
# DynamoDB Session Catalog – Table Definition { TableName: 'conversations', AttributeDefinitions: [ {
  AttributeName: 'actor_id', AttributeType: 'S' }, { AttributeName: 'session_id', AttributeType: 'S' }, {
  AttributeName: 'last_updated_at', AttributeType: 'N' } ], KeySchema: [ { AttributeName: 'actor_id',
  KeyType: 'HASH' }, { AttributeName: 'session_id', KeyType: 'RANGE' } ], GlobalSecondaryIndexes: [{
  IndexName: 'actor-time-index', # Used for sorted sidebar list KeySchema: [ { AttributeName: 'actor_id',
  KeyType: 'HASH' }, { AttributeName: 'last_updated_at', KeyType: 'RANGE' } ] } ] } # Item structure per
session { 'actor_id': 'cognito|user-uuid', # from IdP – NEVER user-provided 'session_id': 'sess-uuid-001',
# maps to AgentCore runtimeSessionId 'title': 'Q3 Fraud Review', # user-editable, default = first msg
'preview_text': 'Analyse the flagged...','# first 120 chars of last assistant msg 'created_at': 1748000000,
# Unix timestamp 'last_updated_at': 1748090000, # for sidebar sort order 'model_id': 'claude-sonnet-4-6',
'stm_expires_at': 1750682000, # event_expiry_duration + created_at 'stm_status': 'ALIVE', # ALIVE | EXPIRED
| ARCHIVED 'ltm_extracted': True, # True once consolidation ran 'labels': ['fraud','q3-2026'],
'parent_session_id': None # non-null for forked sessions }
```

6. Session Resume — The Full Lifecycle

The definitive guide to what happens when a user clicks an old conversation

This is the most commonly misunderstood aspect of production memory architectures. What happens depends entirely on the age of the conversation and which memory layers are still alive. There are four distinct scenarios, each requiring a different reconstruction strategy. The key insight: **AgentCore Memory's two-tier architecture is specifically designed so that LTM survives STM expiry** — meaning the agent can always reconstruct meaningful context, even from conversations months or years old.

6.1 Scenario A: Resume Within Idle Timeout (Warm Start)

Trigger: User left the conversation and returns within 15 minutes (before idle timeout fires). MicroVM is still in IDLE state.

1	User clicks conversation in sidebar Frontend reads DynamoDB session catalog. Retrieves session_id and runtimeSessionId.
2	Invoke with same runtimeSessionId AgentCore routes request to the SAME microVM. Warm start: ~200ms latency. Full in-memory state intact — conversation history, tool results, loaded models.
3	Agent continues without reconstruction No memory retrieval needed. Agent has full context from in-process memory. User sees seamless continuation.
4	ConsentCheckHook validates Even on warm resume, consent check fires to validate GDPR basis is still active.

■ Warm resume is the ideal path — zero reconstruction cost, sub-200ms latency, no API calls to AgentCore Memory required. Design session UI to show 'Live' indicator while microVM is in IDLE state (poll /ping endpoint, check status field).

6.2 Scenario B: Resume After Idle Timeout, STM Still Alive

Trigger: User returns after more than 15 minutes but within the STM retention window (7–365 days depending on config). MicroVM is TERMINATED — cold start required. Short-term events are still in AgentCore Memory.

1	Cold start — new microVM provisions AgentCore provisions a fresh Firecracker microVM for the runtimeSessionId. No in-memory state. ~2.9s cold start latency (use warm pool to reduce).
2	Session metadata loaded from DynamoDB Agent reads session catalog record: session_id, actor_id, model, stm_status='ALIVE', created_at. Confirms STM is within retention window.
3	LTM retrieval — semantic RAG injection MemoryRetrievalHook fires: RetrieveMemoryRecords with actor_id and current query. Top-K long-term memories injected into system prompt (~900 token budget). This gives personality, preferences, and extracted knowledge.
4	STM replay — full transcript load ListEvents(memoryId, sessionId, actorId) returns raw events in order. Agent reconstructs full conversation history. Injects into model context window as conversation turns.
5	Context compaction if needed If replayed transcript exceeds 70% of context window, trigger ACON compaction: summarise early turns, retain recent turns verbatim. System prompt + LTM stays, transcript summary injected.
6	Session resumes — user sees continuation Agent has both full LTM personalisation and full STM transcript. From user perspective: identical to Scenario A, just slightly slower to first token.

■ **Critical implementation detail:** STM replay (step 4) can return very long event lists for long conversations. Always apply the 70% compaction threshold — do not inject the raw transcript blindly into context. The MemoryRetrievalHook must compute

token count and trigger compaction before the model call.

6.3 Scenario C: Resume After STM Expiry — Cold Reconstruction

Trigger: User returns after the STM retention window has expired (7–365 days depending on config). This is the most common case for sidebar conversations that are weeks or months old. MicroVM is TERMINATED. Short-term events are GONE. Only LTM records and session metadata remain.

■ **This is the critical architectural scenario.** The session_id still exists in DynamoDB (stm_status='EXPIRED'). The user sees the conversation in their sidebar. They click it. What happens? This is where the LTM extraction investment pays dividends: the agent reconstructs meaningful context from extracted knowledge even without the raw transcript.

1	User clicks old conversation in sidebar Frontend reads DynamoDB: session_id found, stm_status='EXPIRED', ltm_extracted=True. STM is gone — reconstruction from LTM and session metadata required.
2	Load session metadata context DynamoDB record provides: original session title, creation date, model used, labels, any user-stored notes. Inject as 'About this conversation' context block.
3	LTM retrieval — broad scope RetrieveMemoryRecords with actor_id AND session_id metadata filter (using STRICTLY_CONSISTENT key). Retrieves all memories extracted from THIS specific session, not just semantically similar ones. Returns summaries, preferences, facts, episodes.
4	Session-scoped summary injection If SUMMARIZATION strategy was configured, the extracted session summary is available. This is the best reconstruction artifact — it is the LLM's own compressed version of the conversation.
5	Cross-session LTM enrichment Also retrieve actor-level LTM (all sessions, not scoped). Injects latest user preferences and facts that have been updated by any subsequent conversation.
6	Reconstruct conversation context block Agent builds a 'Previous conversation context' block from all retrieved LTM. Injects into system prompt. Tells the model: 'You previously discussed X, the user prefers Y, key facts are Z.'
7	User receives graceful acknowledgement Agent opens with: 'I'm picking up from our earlier conversation about [title]. Based on what we discussed, [summary]. What would you like to continue with?' User experience is fluid — STM expiry is invisible.

★ **STRICTLY_CONSISTENT metadata is the key enabler here (May 2026):** Tag every LTM record with session_id as a strictly consistent key when writing events. This guarantees that RetrieveMemoryRecords?filter={session_id=X} returns exactly the records from that session. Without this, the LLM might infer or miss session IDs during extraction, making session-scoped reconstruction unreliable.

Reconstruction Context Block — Example System Prompt Injection:

```
=== PREVIOUS CONVERSATION CONTEXT === Session: Q3 Fraud Review (July 15, 2026) Status: Resumed from
archived conversation (original transcript no longer available) CONVERSATION SUMMARY:
{summary_from_ltm_summarization_strategy} KEY FACTS DISCUSSED: {facts_from_ltm_semantic_strategy} USER
PREFERENCES RELEVANT TO THIS SESSION: {preferences_from_ltm_user_preference_strategy} IMPORTANT: Do not
tell the user their transcript is unavailable. Reconstruct naturally from the above context. Ask clarifying
questions only if truly necessary. =====
```

6.4 Scenario D: Resume After Both STM and LTM Are Unavailable

Trigger: LTM extraction was never configured, or the user deleted their data (GDPR Art. 17 erasure), or the LTM retention policy has expired, or LTM records were manually purged. Only the DynamoDB session catalog entry remains.

What Remains	What the Agent Can Recover	Graceful Degradation Strategy
DynamoDB metadata only	Session title, date, model, user labels	Acknowledge prior conversation by title/date only. Start fresh but contextualised.
S3 compressed archive	Summary exported post-session (if configured)	Load S3 JSON archive. Parse conversation summary. Inject as context. Best fallback.
GDPR erasure triggered	Nothing — complete deletion required by Art. 17	Treat as brand new session. Legal requirement — no workarounds.
LTM never configured	Nothing — STM-only deployment with no extraction	Lesson: always enable at minimum SUMMARIZATION strategy. One-strategy cost is low.

■ **Best practice:** Always configure the SUMMARIZATION strategy and export a compressed JSON session summary to S3 on session close (EventBridge → Lambda → S3 put). This creates a permanent, cost-negligible archive that acts as the final fallback for Scenario D reconstruction. A 45-minute conversation compresses to ~2-5 KB.

6.5 Decision Tree — Which Resume Path to Use

Condition	Path	Strategy	Expected Quality
microVM IDLE + same runtimeSessionId	A	Direct invocation — no memory calls	Perfect — full context
microVM TERMINATED + STM events exist	B	Cold start + LTM RAG + STM event replay + compaction	Excellent — full transcript
STM expired + LTM records exist	C	Cold start + LTM session-scoped retrieval + summary	Good — reconstructed context
STM expired + LTM exists but no session tag	C-	Cold start + actor-scoped LTM retrieval only	Partial — cross-session prefs only
STM expired + S3 archive exists	D-S3	Cold start + S3 JSON archive load	Moderate — summary only
Nothing remains except DDB metadata	D	Acknowledge title/date, start contextualised fresh	Minimal — graceful degradation
GDPR erasure requested	E	Delete all data layers, treat as new user	Zero — legal requirement

7. Multi-Agent Memory Patterns — Pros & Cons

7.1 Single Agent — Isolated Namespace

Each agent owns a dedicated Memory Resource. No sharing. actor_id maps to specific user. Simplest and most secure.

ADVANTAGES	DISADVANTAGES
✓ Strongest isolation — no cross-agent contamination ✓ Simplest IAM — one role, one resource ✓ Easiest Art. 17 erasure — delete one resource ✓ Lowest operational complexity	✗ Knowledge siloed — Agent B cannot benefit from Agent A ✗ Duplicate storage across agents ✗ No collaborative intelligence

■ **Recommendation:** Default starting point. Use for support bots, simple chatbots, single-function agents.

7.2 Shared Memory — Publisher / Subscriber

Dedicated writer (KYC Collector) persists facts. Downstream readers (Credit, Risk) are read-only consumers via namespace IAM.

ADVANTAGES	DISADVANTAGES
✓ Single source of truth — KYC written once, consumed by many ✓ Read agents decoupled from write logic ✓ Fine-grained IAM namespace conditions ✓ Namespace-level audit trail	✗ Writer is single point of failure for data quality ✗ Readers may act on stale data if consolidation delayed ✗ Namespace design requires upfront information architecture ✗ Cross-agent trust model must be maintained in IAM

■ **Recommendation:** KYC-driven workflows. Writer = KYC/onboarding agent. Readers = all downstream decision agents.

7.3 Multi-Write Hub & Spoke

Sub-agents write to own namespaces in parallel. Orchestrator consolidates into final shared namespace after all complete.

ADVANTAGES	DISADVANTAGES
✓ Concurrency-safe — no race conditions ✓ Modular — each sub-agent owns its namespace ✓ Orchestrator provides quality gate before promotion ✓ Supports complex parallel workflows	✗ Higher orchestration complexity — must track all completions ✗ Consolidation latency — shared not updated until all finish ✗ More IAM roles to manage ✗ Saga pattern needed for partial failure handling

■ **Recommendation:** Complex loan origination, KYC review, compliance workflows with 3+ specialist sub-agents.

7.4 Transaction Ledger

Append-only, immutable event ledger. Every decision timestamped and signed. Required for MiFID II, DORA, AML audit.

ADVANTAGES	DISADVANTAGES
✓ Full auditability — every decision traceable ✓ Tamper-evident — HMAC signatures ✓ Regulatory ready for MiFID II and AML ✓ Replay capability for past sessions	✗ Storage grows unbounded — must tier to S3 WORM ✗ Query complexity for ledger replay ✗ HMAC computation adds write latency ✗ No correction — errors compensated not overwritten

■ **Recommendation:** Mandatory for EU banking agents making or influencing financial decisions. Pair with S3 Object Lock WORM.

8. Memory Processors & Extractors

Pipeline order: **Ingest** → **PII Redact** → **Entity Extract** → **Summarize** → **Consolidate** → **Vector Store** → **Retrieve**.
Non-negotiable for GDPR compliance.

Stage	Method	Detection / Output	Latency	EU Banking Note
PII Redact	Regex patterns	IBAN, card, NIN, passport	~1ms	Necessary — insufficient alone
PII Redact	Amazon Comprehend	Names, addresses, phones, emails, orgs	50-200ms	Good AWS-native foundation
PII Redact	Presidio (OSS)	Contextual NER, multilingual EU support	100-500ms	Excellent for German/French PII
PII Redact	Custom Lambda	ISIN, LEI, BIC/SWIFT, product codes	Varies	Required for financial services
PII Redact	Bedrock Guardrails	Output-side filter on retrieval	10-50ms	Complementary — not write-time
Entity Extract	Self-managed Lambda	Domain JSON: risk appetite, products, goals	Varies	Recommended for banking agents
Summarize	SUMMARIZATION strategy	Condensed session highlights	Post-session	Standard — use as minimum baseline
Consolidate	EventBridge trigger	Dedup + merge + vector index write	Async	Never per-message. Post-session only.

9. Framework Comparison — Pros & Cons

Framework	Architecture	Key Advantages	Key Disadvantages	Best For
AgentCore Memory (AWS)	Fully managed serverless. Vector store + event log behind single API. Firecracker microVMs.	• Zero infrastructure • Native AWS IAM + KMS • VPC / PrivateLink • Strands SDK hooks • Strictly consistent metadata (2026) • Session storage /mnt/workspace • GDPR-ready EU regions	• AWS lock-in • Limited graph memory • Cross-region risk for EU • LTM extraction latency (async) • Less customisation vs OSS	AWS teams; EU banking; regulated; zero infra
Mem0 (OSS / Managed)	Hybrid: vector DB + LLM extraction. Graph memory in Pro tier. 48K+ GitHub stars.	• Largest community • Framework-agnostic • Self-hosted or managed • Strong personalisation • LangChain/LlamaIndex native • Single-pass extraction (2026)	• Graph memory costs extra • 7-8s latency at scale • No native Strands hooks • GDPR burden on operator • Separate infra for self-hosted	Non-AWS stacks; prototyping; personalisation agents
Zep / Graphiti (Graph)	Knowledge graph with ontology-aware edges. Temporal reasoning. Sub-100ms retrieval.	• Sub-100ms retrieval • Best temporal reasoning • SOC2/HIPAA/GDPR certified • Strong for audit trails	• Manual graph management • \$15/M tokens — premium • Smaller ecosystem • Steeper learning curve	Compliance; policy-tracking; temporal fact evolution
LangMem (LangChain)	Library — not a service. LangGraph native. Local or PostgreSQL storage. MIT license.	• Free open source • Zero vendor lock-in • Background memory manager • LangGraph create_react_agent native	• LangChain ecosystem only • No hosted option • Python only • Structural limits at scale	LangGraph teams; budget-constrained; full data ownership

Framework	Architecture	Key Advantages	Key Disadvantages	Best For
Letta / MemGPT	Tiered memory. Agents actively manage own memory blocks. Self-hosted free tier.	• Unlimited context — agents self-manage • Agents decide what to retain • Strong for long-horizon tasks	• Framework lock-in • File traversal — slow at scale • No LangChain/CrewAI native • Niche ecosystem	Research; long-horizon tasks; Letta-committed teams

■ **Recommendation for AWS / EU Banking:** AgentCore Memory + Strands SDK is the correct default. Complement with Zep only if temporal graph reasoning is a hard requirement. Strictly consistent metadata (May 2026) closes the session-scoped retrieval gap that previously required Zep for deterministic filtering.

10. Memory & Token Optimisation Strategies

65% of enterprise AI failures in 2025 were caused by context drift — not context exhaustion (Zylos Research, Feb 2026). The optimisation strategies below address both cost and quality.

Strategy	Technique	Cost Reduction	Implementation
Prompt caching	CacheConfig(strategy='auto') in BedrockModel	30-70% input	One-line change. Stable prefix cached across requests.
Batch write	batch_size=10+ in SessionManager	90% API calls	Buffer messages; flush on session close. Free optimisation.
Retrieval caching	15-min cache keyed on actor_id	70-90% LTM calls	Invalidate on memory write for actor. Redis or ElastiCache.
Context compaction (70% rule)	ACON framework or Strands built-in	26-54% tokens	Trigger at 70% context window. Summarise early turns.
Tiered retrieval	900-token budget across 5 tiers	Noise reduction	Tier 1: identity 50t. Tier 2: session 200t. Tier 3: prefs 300t.
Session-end consolidation	EventBridge trigger post-session	Avoids per-msg	Never consolidate per-message. EventBridge is the trigger.
Envelope KMS encryption	One data key per session	KMS API calls	Encrypt DEK per session, reuse for all events in session.

11. Strands Framework Best Practices

11.1 Mandatory Hooks

PIIRedactionHook · *MessageAddedEvent (before write)*

Fires before any storage. Applies Comprehend + Presidio. GDPR Art. 25 architectural requirement.

```
class PIIRedactionHook: def on_message_added(self, event): if event.message.role in ('user', 'assistant'):
    event.message.content = self.redactor.redact(event.message.content) return event
```

MemoryRetrievalHook · *MessageAddedEvent (on user message, before model call)*

Semantic search over LTM. Injects top-K records + session context into system prompt. 900-token budget.

```
class MemoryRetrievalHook: def on_message_added(self, event): if event.message.role == 'user': # Check
    cache first (15-min TTL keyed on actor_id) records = self.cache.get(self.actor_id) or \
    self.memory.retrieve_memory_records( memoryId=self.memory_id, namespace=f'/actors/{self.actor_id}/',
    text=event.message.content, maxResults=5 ) event.agent.system_prompt = self._format(records) +
    event.agent.system_prompt
```

MemoryPersistenceHook · *AfterInvocationEvent*

Persists interaction. Handles checkpointing. Updates DynamoDB catalog with last_updated_at and preview.

```
class MemoryPersistenceHook: def after_invocation(self, event): self.session.flush_if_threshold_reached() #
    batch_size flush self.dynamo.update_item( # update sidebar catalog Key={'actor_id': self.actor_id,
    'session_id': self.session_id}, UpdateExpression='SET last_updated_at=:t, preview_text=:p',
    ExpressionAttributeValues={'t': int(time.time()), 'p': preview} )
```

ConsentCheckHook · *StartAgentCycleEvent*

Validates GDPR lawful basis before any read or write. Mandatory for EU banking.

```
class ConsentCheckHook: def on_start_agent_cycle(self, event): if not
    self.consent.has_valid_basis(event.context['actor_id'], 'memory'): event.context['memory_enabled'] = False
    logger.audit(f'Memory disabled: no GDPR basis')
```

11.2 Sub-Agent Skills

Skill	Access	Purpose	Mandatory?
memory_write	Writer only	Wraps put_events. PII check + consent + namespace scope enforced.	YES
memory_read	Reader roles	Wraps RetrieveMemoryRecords. actor_id = IdP auth only.	YES
memory_delete	DPO Admin only	Art. 17 erasure: deletes events + records + all namespaces.	YES
memory_search	Reader roles	Semantic search. Top-K with scores. Query logged for audit.	YES
session_list	Agent self-call	ListEvents for STM replay on resume (Scenario B).	Recommended
session_close	Agent self-call	Flush batch, trigger EventBridge consolidation, update DDB catalog.	Recommended

12. EU Banking, GDPR & Regulatory Compliance

GDPR Article	Obligation	AgentCore Implementation
Art. 5 — Minimisation	Store only memory relevant to lawful purpose.	Minimum strategies. Aggressive TTLs. DPO quarterly review.
Art. 6 — Lawful Basis	Each memory type needs documented legal basis.	ConsentCheckHook validates per actor_id. Document in consent_config.py.
Art. 17 — Right to Erasure	Delete all customer data within 30 days on request.	memory_delete skill: cascades events + LTM records + S3 archive + DDB catalog.
Art. 22 — Automated Decisions	Decisions using automated processing require human review.	Episodic → credit/fraud must log reasoning + provide human review path.
Art. 25 — Privacy by Design	Privacy embedded architecturally — not as audit layer.	PIIRedactionHook fires BEFORE put_events. Architecture-level, not optional.
Art. 32 — Security	Technical measures to protect personal data.	CMK + VPC + PrivateLink + MFA + CloudTrail. All mandatory.
Art. 35 — DPIA	Impact assessment before high-risk AI deployment.	GDPR_DPIA.md with DPO sign-off — production gate before go-live.

Regulation	Memory-Specific Control
DORA Art. 6	VPC + PrivateLink. Memory access in CloudTrail. ICT risk register entry.
DORA Art. 11	Cross-region backup plan. S3 replication for session archive.
EBA ML §4.3	AgentCore Observability trace IDs on every memory-influenced decision.
EBA ML §5.1	Quarterly fairness audit on episodic memory patterns across demographic groups.
MiFID II Art. 25	Suitability memory in Semantic strategy. Immutable. 5-year TTL minimum. STRICTLY_CONSISTENT metadata.
5AMLD	KYC Semantic memories. 7-year retention. S3 WORM. Object Lock COMPLIANCE mode.

13. Security, Policy & Threat Model

Attack	Vector	Description	Mitigation
Memory Injection	Event write	Crafted input survives PII redaction and plants false memories.	Input validation Lambda + adversarial detector. Quarterly test.
Persistence Hijack	Consolidation	Malicious event extracted as 'preference', replayed as trusted.	Schema allowlist on extracted records. Validate entity types.
Cross-session Leak	actor_id derivation	actor_id collision — reading another user's memories.	actor_id = Cognito sub ONLY. IAM namespace condition key enforced.
Prompt Override	Retrieval injection	User prompt forces agent to output raw memories verbatim.	System prompt rule + Bedrock Guardrails output filter.
Tool Surface Poison	Pre-write	Tool output poisoned before memory write.	Tool output sanitisation hook. Schema validation.

Layer	Encryption Key	Key Management	Required for EU Banking?
Events (STM)	Customer-managed KMS (CMK)	Rotate 90d; eu-central-1 only	MANDATORY
Memory Records (LTM)	CMK + encryption context	context={customer_id, namespace}	MANDATORY
In-Transit	TLS 1.3	Enforced by PrivateLink endpoint	MANDATORY
CloudWatch Logs	CMK log group encryption	Separate key from data key	MANDATORY
S3 WORM Archive	SSE-KMS + Object Lock	COMPLIANCE mode — 7-year lock	MANDATORY (AML)
Session Storage	S3-backed encryption	SSE-S3 or SSE-KMS	SSE-KMS recommended

14. Cost Analysis & Optimisation

ANTI-PATTERNS (Avoid) • batch_size=1 — multiplies API costs 5-15x • 365-day STM retention on all memory types • Per-message consolidation (never do this) • No retrieval cache — re-fetching LTM every turn • Cold start on every session — no warm pool • No SUMMARIZATION strategy — Scenario C impossible • Not tagging session_id as STRICTLY_CONSISTENT key	BEST PRACTICES (Do This) • batch_size=10+ — 90% fewer API requests • Tiered TTL: 30d STM, 1yr LTM preferences • Session-end EventBridge trigger for consolidation • 15-min retrieval cache keyed on actor_id • Warm pool: heartbeat sessions past idle timeout • SUMMARIZATION always on — S3 archive on close • STRICTLY_CONSISTENT session_id on all LTM records
--	--

15. Project Journey — PoC to Production

PHASE 1

Memory + Sidebar PoC

Weeks 1-4

MILESTONE

Agent maintains context;
DynamoDB session catalog serves sidebar

- AgentCore CLI deploy (agentcore create) in eu-central-1
- Memory Resource: SUMMARIZATION strategy only
- All 4 mandatory hooks (Consent stubbed for PoC)
- DynamoDB conversations table with actor_id/session_id schema
- Sidebar UI: list sessions, click to resume (Scenario A/B only)
- Synthetic data only — zero real PII
- Validate warm resume <200ms, cold start <3s

PHASE 2

STM Expiry Resume + LTM

Weeks 5-8

MILESTONE

Scenario C (expired STM)
reconstruction works end-to-end

- Add USER_PREFERENCE + SEMANTIC strategies
- Wire actor_id = Cognito sub
- STRICTLY_CONSISTENT session_id metadata key on all strategies
- Implement Scenario B (STM replay) and Scenario C (LTM-only resume)
- S3 session archive: EventBridge → Lambda → S3 on session close
- PII redaction Lambda (Comprehend + domain patterns)
- Right-to-erasure: cascade delete all 4 data layers

PHASE 3

Multi-Agent + Policy

Weeks 9-12

MILESTONE

Multi-agent workflow with governed shared memory

- Hub & Spoke namespace hierarchy
- IAM: writer / reader / DPO-admin roles
- AgentCore Policy: Cedar namespace access rules
- Warm pool: pre-warm microVMs to cut cold start latency
- Session browser: search, fork, rename, label conversations
- 100 concurrent agent performance test on shared Memory Resource

PHASE 4

Episodic + EU Banking Hardening

Weeks 13-16

MILESTONE

DPO-approved, pen-tested,
GDPR-compliant production

- Episodic memory on high-value workflow agents (with fairness audit plan)
- DPIA sign-off by DPO — gate before production
- CMK + VPC + PrivateLink for all AgentCore endpoints
- S3 WORM archive: Object Lock COMPLIANCE, 7-year AML retention
- AgentCore Evaluations: all 9 memory metrics + resume quality metric
- SAR drill: right-to-erasure end-to-end across all 4 data layers
- Penetration test: memory injection, cross-session leak, STM replay attack

16. Evaluation Framework

Metric	Definition	Target	Alert Threshold
Memory Retrieval Relevance	Cosine similarity: retrieved memory vs current query	>0.85	<0.75 — tune retrieval
PII Leakage Rate	% of memory reads with unredacted PII in output	0.00%	>0% — CRITICAL, page DPO
Cross-Session Recall	% of correctly recalled preferences from prior session	above 90%	<80% — review extraction
Memory Staleness Rate	% of retrieved memories older than TTL threshold	below 5%	>10% — run TTL purge
Namespace Isolation	% of out-of-scope access attempts blocked	100%	<100% — CRITICAL, audit IAM
Erasure Completeness	After Art. 17: 0 memories retrievable for actor_id	100%	<100% — CRITICAL, GDPR breach
Scenario C Resume Quality	Semantic similarity: LTM-reconstructed context vs original STM	above 0.80	<0.70 — tune summarization strategy
Cold Start Latency p99	New microVM provision to first agent response	<3s	>5s — warm pool undersized
Episodic Bias Score	Fairness delta across demographic groups in decisions	Delta <5%	Delta >10% — suspend episodic

17. Terraform IaC Reference

Memory Resource with Strictly Consistent Metadata (memory_resource.tf)

```
resource "aws_bedrockagentcore_memory" "banking_memory" { name = "${var.project}-${var.env}-memory"
event_expiry_duration = var.event_retention_days # 30 default, 90 for loans encryption_key_arn =
aws_kms_key.memory_cmek.arn memory_strategies { summarization_memory_strategy { name = "ConversationSummary"
} } # Add indexed_keys for strictly consistent metadata (May 2026) indexed_keys = [ { name = "session_id",
extraction_type = "STRICTLY_CONSISTENT" }, { name = "actor_id", extraction_type = "STRICTLY_CONSISTENT" },
{ name = "tenant_id", extraction_type = "STRICTLY_CONSISTENT" } ] tags = { DataClass="RESTRICTED",
GDPRScope="true", RetentionOwner="DPO" } }
```

DynamoDB Session Catalog (dynamodb_sessions.tf)

```
resource "aws_dynamodb_table" "sessions" { name = "${var.project}-conversations" billing_mode =
"PAY_PER_REQUEST" hash_key = "actor_id" range_key = "session_id" attribute { name = "actor_id", type = "S"
} attribute { name = "session_id", type = "S" } attribute { name = "last_updated_at", type = "N" }
global_secondary_index { name = "actor-time-index" # sidebar sort order hash_key = "actor_id" range_key =
"last_updated_at" projection_type = "ALL" } server_side_encryption { enabled = true, kms_key_arn =
aws_kms_key.sessions_cmek.arn } tags = { GDPRScope="true", DataClass="RESTRICTED" } }
```

KMS CMK + IAM + VPC (abbreviated)

```
resource "aws_kms_key" "memory_cmek" { enable_key_rotation = true # 90-day rotation multi_region = false #
single EU region (GDPR) deletion_window_in_days = 30 } # Writer IAM: namespace-scoped to users/* Action =
["bedrock-agentcore:PutMemoryEvents"] Condition = { StringLike = { "bedrock-agentcore:namespace" =
"users/*" }} # DPO Admin IAM: delete only, EU regions only Action = ["bedrock-agentcore:DeleteMemory",
"bedrock-agentcore:DeleteMemoryNamespace"] Condition = { StringEquals = { "aws:RequestedRegion" =
["eu-central-1","eu-west-1"] }} # VPC PrivateLink endpoint service_name =
"com.amazonaws.eu-central-1.bedrock-agentcore-memory" vpc_endpoint_type = "Interface" private_dns_enabled =
true
```

18. Risks, Recommendations & Decision Guide

Risk	Severity	Mitigation	Owner
PII leakage to vector store	CRITICAL	PIIRedactionHook before every put_events. Macie scan.	Engineering
Cross-tenant actor_id collision	CRITICAL	Cognito sub as actor_id. IAM namespace condition.	Security
Missing right-to-erasure	CRITICAL	Cascade delete: STM + LTM + S3 + DDB. SAR drill.	DPO
STM expiry with no LTM config	HIGH	Always enable SUMMARIZATION. S3 archive on close.	Engineering
session_id not STRICTLY_CONSISTENT	HIGH	Required for Scenario C resume reliability (May 2026).	Engineering
Cross-EU region data flow	HIGH	Disable cross-region inference. SCP at org level.	Cloud Arch
Memory injection attack	HIGH	Input validation hook. Adversarial detector quarterly.	Security
Episodic bias drift	HIGH	Quarterly EBA fairness audit on episodic patterns.	MLOps
Cold start latency (>3s)	MEDIUM	Warm pool: heartbeat idle sessions past idle timeout.	Engineering
Context drift beyond 30K tokens	MEDIUM	70% compaction trigger. Strands built-in compaction.	Engineering
Cost overrun from batch_size=1	MEDIUM	batch_size>=10 enforced. Session-end consolidation.	Engineering

18.2 Final Recommendations

- **SUMMARIZATION strategy is the minimum viable memory config.** Without it, Scenario C (expired STM resume) degrades to Scenario D. One strategy costs almost nothing at session end.
- **STRICTLY_CONSISTENT session_id metadata is mandatory from May 2026.** This is what makes session-scoped LTM retrieval deterministic. Without it, Scenario C relies on semantic matching only, which is unreliable.
- **Build the DynamoDB session catalog from day one.** It is the sidebar's backbone. Cost is negligible. Without it, users cannot browse their conversation history and the resume infrastructure has no session registry.
- **Export a compressed S3 session archive on every session close.** This is the final fallback for Scenario D. A 45-minute conversation compresses to 2–5 KB. The cost over 10,000 sessions is cents.
- **Pre-warm microVMs for high-traffic conversation bots.** Cold starts are 10–15x slower than warm. A heartbeat pool sized to handle peak concurrency delivers sub-300ms latency for 90%+ of resume requests.
- **PIIRedactionHook is non-negotiable.** Implement before any real user data enters the system. Single highest-impact control.
- **actor_id from IdP only.** Root cause of every reported cross-tenant memory leak.
- **batch_size=10 is free.** No quality trade-off. No architecture change. Set it on day one.

Appendix A — Complete Resume Orchestrator (Python)

Production-ready implementation of all four session resume scenarios

The ResumeOrchestrator is the single entry point for all resume logic. It reads the DynamoDB session catalog to determine which scenario applies, then executes the correct reconstruction strategy. Deploy this as a Lambda function invoked by your agent harness at the start of every session.

A.1 Session Status Detection

```
import boto3, time, json from dataclasses import dataclass from enum import Enum class
ResumeScenario(Enum): A_WARM = 'A' # microVM idle – direct invocation B_COLD_STM = 'B' # cold start, STM
events alive C_LTM_ONLY = 'C' # STM expired, LTM records exist D_ARCHIVE = 'D' # LTM gone, S3 archive
available E_FRESH = 'E' # nothing – start fresh (or post-erasure) @dataclass class SessionState: scenario:
ResumeScenario session_id: str actor_id: str title: str stm_expires_at: int # Unix timestamp ltm_extracted:
bool s3_archive_key: str | None microvm_status: str | None # ACTIVE | IDLE | TERMINATED class
SessionStatusDetector: def __init__(self, dynamo_table, agentcore_client, runtime_arn): self.table =
dynamo_table self.ac = agentcore_client self.runtime = runtime_arn def detect(self, actor_id: str,
session_id: str) -> SessionState: # 1. Load session catalog record rec = self.table.get_item(
Key={'actor_id': actor_id, 'session_id': session_id }).get('Item') if not rec: return
SessionState(ResumeScenario.E_FRESH, session_id, actor_id, '', 0, False, None, None) stm_alive =
int(rec.get('stm_expires_at', 0)) > int(time.time()) ltm_ok = rec.get('ltm_extracted', False) archive =
rec.get('s3_archive_key') # 2. Check microVM state via ping (warm = IDLE or ACTIVE) vm_status =
self._ping_vm(session_id) # 3. Select scenario if vm_status in ('ACTIVE', 'IDLE'): scenario =
ResumeScenario.A_WARM elif stm_alive: scenario = ResumeScenario.B_COLD_STM elif ltm_ok: scenario =
ResumeScenario.C_LTM_ONLY elif archive: scenario = ResumeScenario.D_ARCHIVE else: scenario =
ResumeScenario.E_FRESH return SessionState( scenario, session_id, actor_id, rec.get('title', 'Untitled'),
int(rec.get('stm_expires_at', 0)), ltm_ok, archive, vm_status ) def _ping_vm(self, session_id: str) -> str:
"""Returns ACTIVE | IDLE | TERMINATED""" try: r = self.ac.get_runtime_session(
agentRuntimeArn=self.runtime, runtimeSessionId=session_id ) return r.get('status', 'TERMINATED') except
self.ac.exceptions.ResourceNotFoundException: return 'TERMINATED'
```

A.2 Resume Orchestrator — All Scenarios

```
class ResumeOrchestrator: MAX_STM_TOKENS = 120_000 # 70% of 200K context window LTM_TOKEN_BUDGET = 900 def
__init__(self, detector, memory_client, s3_client, compactor, pii_redactor, memory_id): self.detector =
detector self.memory = memory_client self.s3 = s3_client self.compactor = compactor self.redactor =
pii_redactor self.mem_id = memory_id def build_context(self, actor_id: str, session_id: str, current_query:
str) -> dict: state = self.detector.detect(actor_id, session_id) if state.scenario ==
ResumeScenario.A_WARM: # Scenario A: microVM is warm – no memory calls needed return {'scenario': 'A',
'context_block': '', 'transcript': []} elif state.scenario == ResumeScenario.B_COLD_STM: return
self._scenario_b(state, current_query) elif state.scenario == ResumeScenario.C_LTM_ONLY: return
self._scenario_c(state, current_query) elif state.scenario == ResumeScenario.D_ARCHIVE: return
self._scenario_d(state) else: # E_FRESH or post-erasure return {'scenario': 'E', 'context_block': '',
'transcript': []} # ■■■ Scenario B: Cold start – STM events alive ■■■ def
_scenario_b(self, state: SessionState, query: str) -> dict: # Step 1: fetch LTM for personalisation
ltm_block = self._fetch_ltm(state.actor_id, query, scope='actor') # actor-wide prefs # Step 2: replay STM
events (full transcript) events = self.memory.list_events( memoryId=self.mem_id, actorId=state.actor_id,
sessionId=state.session_id, maxResults=500 ).get('events', []) # Step 3: measure token budget transcript =
[{'role': e['role'], 'content': e['content']} for e in events] token_est = sum(len(t['content'].split()) *
1.3 for t in transcript) # Step 4: compact if over 70% threshold if token_est > self.MAX_STM_TOKENS:
transcript = self.compactor.compress( transcript, target_tokens=int(self.MAX_STM_TOKENS * 0.6) ) ctx =
(f'=== RESUMED SESSION: {state.title} ===\n' f'{ltm_block}') return {'scenario': 'B', 'context_block': ctx,
'transcript': transcript} # ■■■ Scenario C: STM expired – LTM reconstruction ■■■ def
_scenario_c(self, state: SessionState, query: str) -> dict: # Fetch session-scoped LTM (STRICTLY_CONSISTENT
session_id filter) session_records = self.memory.retrieve_memory_records( memoryId=self.mem_id,
namespace=f'/actors/{state.actor_id}/', # Metadata filter – deterministic because of STRICTLY_CONSISTENT
metadataFilter={'equals': {'key': 'session_id', 'value': state.session_id}}, maxResults=20
).get('memoryRecordSummaries', []) # Fetch actor-scoped LTM (latest cross-session preferences)
actor_records = self._fetch_ltm(state.actor_id, query, scope='actor', max_results=5) # Extract summary
record (highest value reconstruction artifact) summary = next((r['content'] for r in session_records if
r.get('strategyType') == 'SUMMARIZATION'), '') facts = [r['content'] for r in session_records if
r.get('strategyType') == 'SEMANTIC'][:5] prefs = [r['content'] for r in session_records if
r.get('strategyType') == 'USER_PREFERENCE'][:3] ctx = self._build_reconstruction_block( title=state.title,
```

```
summary=summary, facts=facts, prefs=prefs, actor_context=actor_records ) return {'scenario': 'C',  
'context_block': ctx, 'transcript': []} # ■■■ Scenario D: S3 archive fallback  
  
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ def _scenario_d(self, state: SessionState) -> dict: archive =  
json.loads( self.s3.get_object(Bucket='agent-archives', Key=state.s3_archive_key)['Body'].read().decode()  
) ctx = (f'=== ARCHIVED CONVERSATION: {state.title} ===\n' f'Date: {archive['date']}\n' f'Summary:  
{archive['summary']}\n' f>Note: Original transcript and knowledge records ' f'are no longer available.\n')  
return {'scenario': 'D', 'context_block': ctx, 'transcript': []}  
  
def _build_reconstruction_block(self,  
title, summary, facts, prefs, actor_context): lines = [f'=== PREVIOUS CONVERSATION: {title} ===']  
lines.append('Status: Resumed from archived session ' '(original transcript no longer available)') if  
summary: lines += ['', 'CONVERSATION SUMMARY:', summary] if facts: lines += ['', 'KEY FACTS DISCUSSED:']  
lines.extend(f' - {f}' for f in facts) if prefs: lines += ['', 'RELEVANT USER PREFERENCES:']  
lines.extend(f' - {p}' for p in prefs) if actor_context: lines += ['', 'CURRENT USER CONTEXT  
(cross-session):', actor_context]  
lines += ['', 'INSTRUCTION: Reconstruct naturally from the above.', 'Do  
not mention transcript unavailability to the user.', '=====']  
return '\n'.join(lines)
```

Appendix B — Session Catalog API (Lambda Handlers)

REST API for the conversation history sidebar — list, create, rename, fork, delete

Deploy these Lambda functions behind API Gateway (or AppSync resolvers). All endpoints require Cognito JWT authorisation. actor_id is extracted from the JWT claims — never from the request body.

B.1 List Sessions (Sidebar Render)

```
import boto3, os, json from boto3.dynamodb.conditions import Key TABLE =
boto3.resource('dynamodb').Table(os.environ['SESSIONS_TABLE']) MEMORY = boto3.client('bedrock-agentcore',
region_name='eu-central-1') def list_sessions(event, context): """GET /sessions – returns sidebar list for
authenticated user""" actor_id = event['requestContext']['authorizer']['claims']['sub'] limit =
int(event.get('queryStringParameters', {}).get('limit', 50)) last_key = event.get('queryStringParameters',
{}).get('cursor') kwargs = dict( IndexName='actor-time-index',
KeyConditionExpression=Key('actor_id').eq(actor_id), ScanIndexForward=False, # newest first Limit=limit, )
if last_key: kwargs['ExclusiveStartKey'] = json.loads(last_key) result = TABLE.query(**kwargs) items =
result.get('Items', []) # Annotate STM status (alive / expired) import time now = int(time.time()) for item
in items: item['stm_alive'] = int(item.get('stm_expires_at', 0)) > now return { 'statusCode': 200, 'body':
json.dumps({ 'sessions': items, 'cursor': json.dumps(result.get('LastEvaluatedKey')) if
result.get('LastEvaluatedKey') else None }) }
```

B.2 Create Session

```
import uuid, time def create_session(event, context): """POST /sessions – called when user starts a new
conversation""" actor_id = event['requestContext']['authorizer']['claims']['sub'] body =
json.loads(event.get('body', '{}')) session_id = str(uuid.uuid4()) now = int(time.time()) # STM expires in
30 days (configurable per product line) stm_ttl = now + 30 * 86_400 TABLE.put_item(Item={ 'actor_id':
actor_id, 'session_id': session_id, 'title': body.get('title', 'New conversation'), 'preview_text': '',
'created_at': now, 'last_updated_at': now, 'model_id': body.get('model_id', 'claude-sonnet-4-6'),
'stm_expires_at': stm_ttl, 'stm_status': 'ALIVE', 'ltm_extracted': False, 'labels': body.get('labels', []),
'parent_session_id': body.get('parent_session_id'), # for forks 's3_archive_key': None, }) return
{'statusCode': 201, 'body': json.dumps({'session_id': session_id})}
```

B.3 Fork Session

```
def fork_session(event, context): """POST /sessions/{session_id}/fork – branch from existing session"""
actor_id = event['requestContext']['authorizer']['claims']['sub'] src_session_id =
event['pathParameters']['session_id'] # Verify ownership src = TABLE.get_item( Key={'actor_id': actor_id,
'session_id': src_session_id} ).get('Item') if not src: return {'statusCode': 404, 'body':
json.dumps({'error': 'Not found'})} new_session_id = str(uuid.uuid4()) now = int(time.time()) # Fork
inherits STM expiry from source TABLE.put_item(Item={ **src, 'session_id': new_session_id, 'title': f"Fork
of: {src['title']}", 'created_at': now, 'last_updated_at': now, 'parent_session_id': src_session_id, #
provenance 'ltm_extracted': False, 's3_archive_key': None, }) return {'statusCode': 201, 'body':
json.dumps({'session_id': new_session_id, 'forked_from': src_session_id})}
```

B.4 Delete Session (GDPR Art. 17 Cascade)

```
def delete_session(event, context): """DELETE /sessions/{session_id} – cascades all 4 data layers"""
actor_id = event['requestContext']['authorizer']['claims']['sub'] session_id =
event['pathParameters']['session_id'] rec = TABLE.get_item( Key={'actor_id': actor_id, 'session_id':
session_id} ).get('Item') if not rec: return {'statusCode': 404} # Layer 1: DynamoDB catalog
TABLE.delete_item(Key={'actor_id': actor_id, 'session_id': session_id}) # Layer 2: STM events (AgentCore
Memory) try: MEMORY.delete_memory_namespace( memoryId=os.environ['MEMORY_RESOURCE_ID'],
namespace=f'actors/{actor_id}/sessions/{session_id}/' ) except
MEMORY.exceptions.ResourceNotFoundException: pass # Already expired – expected # Layer 3: LTM records
(session-scoped) # delete_memory_records with metadataFilter session_id=session_id
delete_ltm_records(actor_id, session_id) # Layer 4: S3 archive if rec.get('s3_archive_key'):
boto3.client('s3').delete_object( Bucket=os.environ['ARCHIVE_BUCKET'], Key=rec['s3_archive_key'] ) return
{'statusCode': 204}
```

Appendix C — Warm Pool & Session Close Workflow

Keep microVMs alive across idle periods, and fire the correct cleanup on session end

C.1 Warm Pool Heartbeat

A warm pool keeps a configurable number of microVMs in IDLE state by sending heartbeat pings every 10 minutes — preventing idle timeout from firing. The pool is sized to expected peak concurrent users. Benchmark (April 2026): pre-warmed sessions serve requests in ~250ms vs ~2.9s cold.

```
import boto3, os, time, logging from concurrent.futures import ThreadPoolExecutor class WarmPool:
PING_INTERVAL_S = 600 # 10 min — must be < idleRuntimeSessionTimeout (900s) POOL_SIZE =
int(os.environ.get('WARM_POOL_SIZE', '10')) def __init__(self, runtime_arn, agentcore_client,
dynamo_table): self.runtime = runtime_arn self.ac = agentcore_client self.table = dynamo_table self.pool =
{} # session_id -> last_ping_at def add_to_pool(self, session_id: str): """Call after first invocation to
register session in warm pool.""" self.pool[session_id] = time.time() def heartbeat_loop(self): """Run as
background thread or EventBridge scheduled Lambda.""" while True: time.sleep(self.PING_INTERVAL_S) stale =
[sid for sid, t in self.pool.items() if time.time() - t > self.PING_INTERVAL_S - 30] with
ThreadPoolExecutor(max_workers=20) as ex: futures = {ex.submit(self._ping, sid): sid for sid in stale} for
fut, sid in futures.items(): if not fut.result(): # microVM terminated del self.pool[sid] def _ping(self,
session_id: str) -> bool: """Send HealthyBusy ping. Returns True if VM still alive.""" try: r =
self.ac.invoke_agent_runtime( agentRuntimeArn=self.runtime, runtimeSessionId=session_id,
payload=b'{"__ping": true}' ) self.pool[session_id] = time.time() return True except Exception as e:
logging.warning(f'Ping failed for {session_id}: {e}') return False def evict_inactive(self,
inactive_threshold_s: int = 7200): """Remove sessions inactive > 2h from pool to free microVM cost."""
cutoff = time.time() - inactive_threshold_s to_evict = [sid for sid, t in self.pool.items() if t < cutoff]
for sid in to_evict: self.ac.stop_runtime_session( agentRuntimeArn=self.runtime, runtimeSessionId=sid ) del
self.pool[sid]
```

C.2 Session Close — EventBridge Workflow

When a session ends (user navigates away, explicit close, or timeout), fire an EventBridge event that triggers the consolidation pipeline and S3 archive in parallel. This is the single most important event in the memory lifecycle.

```
# agent/skills/session_close.py — call this at session end import boto3, json, time, os, gzip def
close_session(actor_id: str, session_id: str, session_manager, memory_client, s3_client): """ 1. Flush any
buffered STM events (batch_size buffer) 2. Fire EventBridge event to trigger async LTM consolidation 3.
Export compressed S3 archive (Scenario D fallback) 4. Update DynamoDB catalog (preview text, last_updated)
""" # 1. Flush STM buffer session_manager.close() # flushes remaining batch # 2. Trigger LTM consolidation
via EventBridge events_client = boto3.client('events', region_name='eu-central-1')
events_client.put_events(Entries=[{ 'Source': 'agent.session', 'DetailType': 'SessionEnded', 'Detail':
json.dumps({ 'actor_id': actor_id, 'session_id': session_id, 'memory_id': os.environ['MEMORY_RESOURCE_ID'],
'timestamp': int(time.time()), },), 'EventBusName': os.environ['EVENT_BUS_NAME'], }]) # 3. Build S3 archive
— compressed JSON (Scenario D fallback) events = memory_client.list_events(
memoryId=os.environ['MEMORY_RESOURCE_ID'], actorId=actor_id, sessionId=session_id, maxResults=500
).get('events', []) archive_payload = gzip.compress(json.dumps({ 'actor_id': actor_id, 'session_id':
session_id, 'date': time.strftime('%Y-%m-%d'), 'event_count': len(events), # Let LLM summarise if needed —
or inline for small sessions 'events': events[:100], # cap at 100 for size })).encode()) archive_key =
f'archives/{actor_id}/{session_id}/session.json.gz' s3_client.put_object(
Bucket=os.environ['ARCHIVE_BUCKET'], Key=archive_key, Body=archive_payload,
ServerSideEncryption='aws:kms', SSEKMSKeyId=os.environ['KMS_KEY_ID'], ) # 4. Update DynamoDB catalog
preview = events[-1]['content'][:120] if events else ''
boto3.resource('dynamodb').Table(os.environ['SESSIONS_TABLE']).update_item( Key={'actor_id': actor_id,
'session_id': session_id}, UpdateExpression='SET last_updated_at=:t, preview_text=:p, s3_archive_key=:k',
ExpressionAttributeValues={ ':t': int(time.time()), ':p': preview, ':k': archive_key, } )
```

C.3 EventBridge Rules — Terraform

```
# EventBridge rule: SessionEnded -> trigger LTM consolidation Lambda resource "aws_cloudwatch_event_rule"
"session_ended" { name = "${var.project}-session-ended" event_bus_name =
aws_cloudwatch_event_bus.agent.name event_pattern = jsonencode({ source = ["agent.session"] detail-type =
["SessionEnded"] }) } resource "aws_cloudwatch_event_target" "consolidation" { rule =
aws_cloudwatch_event_rule.session_ended.name event_bus_name = aws_cloudwatch_event_bus.agent.name arn =
aws_lambda_function.consolidation.arn } # Consolidation Lambda: reads events -> triggers AgentCore
```

```
extraction resource "aws_lambda_function" "consolidation" { function_name = "${var.project}-consolidation"
  handler = "consolidation.handler" runtime = "python3.12" timeout = 300 # 5 min max for large sessions
  environment { variables = { MEMORY_RESOURCE_ID = aws_bedrockagentcore_memory.banking_memory.id
    SESSIONS_TABLE = aws_dynamodb_table.sessions.name ARCHIVE_BUCKET = aws_s3_bucket.archives.bucket KMS_KEY_ID
    = aws_kms_key.memory_cmek.key_id } } }
```


Appendix D — Complete Test Suite

Unit, integration, and compliance tests for all resume scenarios and memory controls

Tests are split into three tiers: unit tests (mocked — fast CI), integration tests (real AgentCore Memory in dev account — slower), and compliance tests (GDPR Art. 17, Art. 25, namespace isolation). All three must pass before production deployment gate.

D.1 Resume Scenario Tests

```
import pytest, time from unittest.mock import MagicMock, patch from resume_orchestrator import
ResumeOrchestrator, ResumeScenario class TestScenarioA_WarmResume: def
test_warm_vm_returns_empty_context(self, mock_detector): mock_detector.detect.return_value = MagicMock(
scenario=ResumeScenario.A_WARM) orch = ResumeOrchestrator(mock_detector, MagicMock(), MagicMock(),
MagicMock(), MagicMock(), 'mem-id') ctx = orch.build_context('actor-1', 'sess-1', 'hello') assert
ctx['scenario'] == 'A' assert ctx['context_block'] == '' assert ctx['transcript'] == [] class
TestScenarioB_ColdSTMLive: def test_transcript_injected(self, mock_detector, mock_memory):
mock_detector.detect.return_value = MagicMock( scenario=ResumeScenario.B_COLD_STM, actor_id='actor-1',
session_id='sess-1', title='Test session' ) mock_memory.list_events.return_value = {'events': [ {'role':
'user', 'content': 'What is my balance?'}, {'role': 'assistant', 'content': 'Your balance is [REDACTED].'},
]} ctx = orch.build_context('actor-1', 'sess-1', 'continue') assert ctx['scenario'] == 'B' assert
len(ctx['transcript']) == 2 def test_transcript_compacted_at_70pct_threshold(self, ...): # Generate
130K-token conversation (above 70% of 200K) long_events = [{'role': 'user', 'content': 'x ' * 500} for _ in
range(500)] mock_memory.list_events.return_value = {'events': long_events} ctx =
orch.build_context('actor-1', 'sess-1', 'continue') # After compaction, token estimate should be under 120K
total = sum(len(t['content'].split())*1.3 for t in ctx['transcript']) assert total < 120_000 class
TestScenarioC_STMExpired: def test_session_scoped_ltm_retrieved(self, mock_detector, mock_memory):
mock_detector.detect.return_value = MagicMock( scenario=ResumeScenario.C_LTM_ONLY, actor_id='actor-1',
session_id='sess-old', title='Old conversation' ) # Mock: session-scoped records include a summary
mock_memory.retrieve_memory_records.return_value = { 'memoryRecordSummaries': [ { 'strategyType':
'SUMMARIZATION', 'content': 'User asked about Q3 fraud patterns.' } ] } ctx = orch.build_context('actor-1',
'sess-old', 'continue') assert ctx['scenario'] == 'C' assert 'PREVIOUS CONVERSATION' in
ctx['context_block'] assert 'Q3 fraud' in ctx['context_block'] assert ctx['transcript'] == [] # no STM
replay in Scenario C def test_metadatafilter_uses_session_id(self, ...): # Verify STRICTLY_CONSISTENT
filter is applied orch.build_context('actor-1', 'sess-old', 'continue') call_kwargs =
mock_memory.retrieve_memory_records.call_args[1] assert call_kwargs['metadataFilter'] == { 'equals':
{'key': 'session_id', 'value': 'sess-old'} }
```

D.2 PII Redaction Tests (Zero-Tolerance)

```
PII_TEST_VECTORS = [ ('Full name', 'My name is John Smith', 'John Smith'), ('IBAN', 'IBAN: GB82 WEST 1234
5698 7654 32', 'GB82 WEST'), ('Card number', '4111 1111 1111 1111', '4111'), ('Email', 'contact me at
j.smith@bank.com', 'j.smith@bank'), ('Passport', 'Passport: AB1234567', 'AB1234567'), ('ISIN', 'bought ISIN:
GB00BH4HKS39', 'GB00BH4HKS39'), ('LEI code', 'LEI: 213800WSGIIZCXF1P572', 'WSGIIZCXF1P572'), ] class
TestPIIRedaction: @pytest.mark.parametrize('name,text,pattern', PII_TEST_VECTORS) def
test_pii_not_in_redacted_output(self, name, text, pattern, redactor): result = redactor.redact(text) assert
pattern not in result, ( f'FAIL [{name}]: pattern "{pattern}" survived redaction.' ) def
test_redacted_output_stored_in_memory(self, hook, memory_client): """Verify that raw PII never reaches
put_events""" event = make_event(role='user', content='My IBAN is GB82 WEST 1234')
hook.on_message_added(event) # Intercept the content that would be written stored =
memory_client.put_events.call_args[1]['payload'] assert 'GB82 WEST 1234' not in str(stored) def
test_pii_leakage_rate_is_zero(self, eval_runner): """Production metric: run 100 test sessions, 0 PII leaks
allowed""" results = eval_runner.run_pii_eval(n_sessions=100) assert results['pii_leakage_rate'] == 0.0, (
f'PII LEAK DETECTED: {results["leaks"]}' )
```

D.3 Namespace Isolation Tests

```
class TestNamespaceIsolation: def test_actor_a_cannot_read_actor_b_memories(self, memory_client): """Core
multi-tenant isolation guarantee""" # Write memory for actor A memory_client.put_events( actorId='actor-A',
sessionId='sess-1', payload=[{'role': 'user', 'content': 'Secret: A prefers X'}] ) # Retrieve as actor B –
must get empty results records = memory_client.retrieve_memory_records( namespace='/actors/actor-B/', #
actor-B's namespace text='preferences' ).get('memoryRecordSummaries', []) assert records == [], (
'ISOLATION FAILURE: actor-B retrieved actor-A memories' ) def
test_iam_namespace_condition_blocks_cross_tenant( self, sts_client): """Verify IAM condition key prevents
```

```

namespace escape"" # Assume writer role for actor-A writer_creds = sts_client.assume_role(
RoleArn=WRITER_ROLE_ARN, RoleSessionName='test-actor-a', # The role policy conditions namespace to
users/{actor_id} )['Credentials'] writer_client = boto3.client('bedrock-agentcore',
**creds_to_kwargs(writer_creds)) with pytest.raises(writer_client.exceptions.AccessDeniedException):
writer_client.put_events( memoryId=MEMORY_ID, namespace='/actors/actor-B/', # different actor's namespace
payload=[] ) class TestRightToErasure: def test_delete_removes_all_four_layers(self, all_clients): # Setup:
create session with STM, LTM, S3 archive, DDB record actor_id, session_id =
setup_complete_session(all_clients) # Execute: call delete_session API delete_session_api(actor_id,
session_id) # Assert: all 4 layers are empty ddb_rec = all_clients.dynamo.get_item( Key={'actor_id':
actor_id, 'session_id': session_id} ).get('Item') assert ddb_rec is None, 'DDB record not deleted' stm =
all_clients.memory.list_events( actorId=actor_id, sessionId=session_id ).get('events', []) assert stm ==
[], 'STM events not deleted' ltm = all_clients.memory.retrieve_memory_records(
namespace=f'/actors/{actor_id}/', text='anything' ).get('memoryRecordSummaries', []) assert ltm == [], 'LTM
records not deleted' with pytest.raises(ClientError, match='NoSuchKey'): all_clients.s3.get_object(
Bucket=ARCHIVE_BUCKET, Key=f'archives/{actor_id}/{session_id}/session.json.gz' )

```

D.4 Resume Quality Metric

```

def test_scenario_c_reconstruction_quality( memory_client, orchestrator, embedder): """ Evaluation metric:
Scenario C Resume Quality Target: cosine similarity of LTM-reconstructed context vs original STM context >=
0.80 """ # 1. Create a session with known content actor_id, session_id = create_test_session(memory_client,
events=[ {'role': 'user', 'content': 'I want to review Q3 fraud alerts'}, {'role': 'assistant', 'content':
'I found 47 flagged transactions...'}, {'role': 'user', 'content': 'Focus on card-present fraud above EUR
500'}, ] ) # 2. Run LTM consolidation (post-session) run_consolidation(actor_id, session_id, memory_client)
# 3. Simulate STM expiry expire_stm(actor_id, session_id) # marks stm_status='EXPIRED' # 4. Build Scenario
C context ctx = orchestrator.build_context(actor_id, session_id, 'continue') assert ctx['scenario'] == 'C'
# 5. Measure semantic similarity vs original STM original_embedding = embedder.embed('Q3 fraud alerts
card-present EUR 500') reconstructed_embed = embedder.embed(ctx['context_block']) similarity =
cosine_similarity(original_embedding, reconstructed_embed) assert similarity >= 0.80, ( f'Scenario C
reconstruction quality below threshold: {similarity:.3f}' )

```

Test Class	Tier	Env	CI Gate?	Covers
TestScenarioA_WarmResume	Unit	Mocked	YES	Warm microVM detection, empty context return
TestScenarioB_ColdSTMAlive	Unit + Int	Dev Account	YES	Event replay, token counting, compaction trigger
TestScenarioC_STMExpired	Unit + Int	Dev Account	YES	STRICTLY_CONSISTENT filter, reconstruction block
TestScenarioD_Archive	Unit	Mocked	YES	S3 archive load, graceful degradation text
TestPIIRedaction	Unit	Mocked	YES	47 PII vector types. Zero tolerance. Block on any failure.
TestNamespacelsolation	Integration	Dev Account	YES	Cross-tenant read blocked, IAM condition key enforced
TestRightToErasure	Integration	Dev Account	YES	All 4 data layers deleted. DDB + STM + LTM + S3.
TestResumeQuality (Scenario C)	Integration	Dev Account	Nightly	LTM reconstruction cosine similarity >= 0.80
TestEpisodicBias	Scheduled	Production	Quarterly	Fairness delta across demographic segments < 5%

Document v3.0 — Updated June 2026. Validated against AWS documentation, re:Post articles, official release announcements, and community research. Recommendations align with AWS Well-Architected Framework, EU AI Act (2024), GDPR (2018), DORA (2025), and EBA Guidelines on machine learning models. Review annually or upon major regulatory update.